

Today Todo · 오늘 할 일과 하루 기록

Flutter와 Riverpod으로 만든 네이티브 To Do 앱입니다. 목록·추가·삭제에 카테고리별 장소, 예상시간, 완료 수행점수, 날짜별 달력, 로컬 기록과 회고를 더했습니다. 이 문서는 **2.0.0+2 확장 버전**의 실행 방법과 구현 구조, 제출 증빙을 설명합니다.

먼저 실행하기

이 컴퓨터는 Flutter SDK와 Windows C++ 빌드 도구를 준비했습니다. 프로젝트 폴더의 **앱 실행.cmd**를 더블클릭하면 제공한 release 앱을 실행합니다. 최종 실행용 배포 위치는 **output/windows/Release/**이며, 깨끗한 소스 복사본에서 빌드한 11개 파일을 원본과 같은 SHA256으로 복사했습니다. 실행 파일이 없거나 소스를 수정했다면 다음 명령으로 빌드합니다.

```
powershell -ExecutionPolicy Bypass -File .\scripts\run_app.ps1 -Build
```

배포 실행 파일은 **output/windows/Release/today_todo.exe**입니다. 직접 빌드하면

build/windows/x64/runner/Release/today_todo.exe가 생성됩니다. 실행 스크립트는 제공한 배포 폴더를 우선 사용하고 일반 빌드 폴더를 대체 경로로 사용하도록 구성합니다. 소스를 다시 빌드한 결과를 바로 확인하려면 그 빌드 폴더의 exe를 직접 실행합니다.

exe만 따로 옮기면 필요한 DLL·data가 빠질 수 있으므로 실행용으로 옮길 때에는 **Release 폴더 전체**를 함께 복사합니다. 과제 제출 ZIP에는 의존성·빌드 결과를 빼고 소스를 넣습니다.

개발 화면을 실행하거나 DevTools를 사용할 때에는 프로젝트 루트에서 아래 명령을 사용합니다.

```
powershell -ExecutionPolicy Bypass -File .\scripts\run_app.ps1
powershell -ExecutionPolicy Bypass -File .\scripts\run_app.ps1 -Profile
```

첫 명령은 debug, 두 번째 명령은 profile 실행입니다. 다른 모드로 바꾸려면 실행 터미널에서 q로 종료한 뒤 다음 명령을 실행합니다. 실행 스크립트는 로컬 SDK 또는 PATH의 Flutter를 찾고 의존성 설치·Windows 플러그인 폴더 준비 후 실행합니다. **-ExecutionPolicy Bypass**는 해당 PowerShell 프로세스에 적용되며 시스템 정책을 바꾸지 않습니다.

현재 검증 상태: 전체 테스트 **56개**, 깨끗한 소스 복사본의 분석·release 빌드, 실제 Windows 종료/재시작 저장 복원이 통과했습니다. 공부·도서관·45분·9점 항목과 회고가 복원됐고 하루 점수 3.8·장소 관측 100%(1/1)을 다시 확인했습니다. 기능 14장·DevTools 4장, **전체 18장 원본 검토와 최종 release 실행을 완료**했습니다. PDF 31페이지의 시각 검토와 제출 ZIP의 필수 파일·CRC·해시·의존성 제외 검사도 통과했습니다.

1. 과제 요구사항과 확장 기능

구분	요구사항	구현 증빙
기본 기능 20점	리스트·추가·삭제	날짜별 목록, 입력 폼, 고유 ID 삭제와 전후 캡처
보너스 10점	Riverpod	같은 앱의 비동기 상태를 AsyncNotifierProvider로 관리
보너스 10점	네이티브 DevTools	Windows 앱의 Inspector·Timeline·Memory·Performance
코드 제출	빌드·실행 가능한 자체 코드	소스·의존성 선언·플랫폼 설정·테스트
문서 제출	Readme.pdf	실행 명령·구조·자료구조·보너스·실제 이미지
확장	장소·예상시간·완료 상태	항목 추가·수정·완료 평점 입력
확장	0~10 수행점수·활동 평점	품질 평균과 양을 반영한 하루 점수 구분
확장	달력·영구 기록	날짜별 기록과 로컬 저장·재시작 복원
확장	카테고리별 장소·자동완성	공부·운동·사용자 카테고리에 장소 목록 보관
확장	장소 관찰 통계	8점 이상 완료 비율·표본 수·완료 평균·완료 예상분
확장	목록 분리·회고	해야 할 일/실제로 한 일, 날짜별 잘한 점·아쉬운 점·개선할 점

리스트·추가·삭제 세 기능의 합계가 20점입니다. 보너스는 같은 앱에 적용합니다. 마감은 **2026년 10월 11일 일요일 23:59 KST**입니다.

일반 앱은 사용자가 입력한 기록을 로컬 저장소에 보관합니다. 촬영용 과거 3일의 샘플은 별도 메모리 저장소에 주입하며 일반 앱 데이터와 섞지 않습니다. 서버·로그인·클라우드 동기화는 구현 범위에 포함하지 않습니다.

2. 개발 환경

다음 버전은 실제 SDK·doctor·pubspec.lock에서 확인한 작업공간 환경입니다. 이 환경 정보와 v2 앱의 최종 통과 결과는 구분합니다.

항목	확인한 값
OS	Microsoft Windows 10.0.26200.9457, locale ko-KR
Flutter	3.47.6 stable, revision 5fc346839b
Dart	3.13.5
flutter_riverpod / riverpod	3.4.3 / 3.4.3
shared_preferences	2.5.6, pubspec.lock 기준
DevTools	2.60.0
Visual Studio	Build Tools 2022 17.14.41, 설치 버전 17.14.37710.0

항목	확인한 값
Windows SDK	Windows 10 SDK 10.0.26100.0
검증 대상	Windows x64 네이티브
Android	SDK 미설치, 선택적 대상으로 실행·빌드 미검증

다른 컴퓨터에서는 다음 명령으로 자신의 환경을 확인합니다.

```
flutter --version
dart --version
flutter doctor -v
flutter devices
```

최종 앱 검증 로그는 [docs/evidence/v2_*.txt](#)에 보관합니다. 최종 요약은 [docs/evidence/validation_v2.json](#)에 기록하고 제출에는 최신 증빙만 선별합니다.

3. 다른 컴퓨터의 최초 환경 준비

현재 컴퓨터에서 실행하는 경우 위의 쉬운 실행 방법을 사용합니다. 새 컴퓨터에서 제출 소스를 빌드할 때에만 이 절을 따릅니다.

1. Git for Windows와 Windows stable Flutter SDK를 설치합니다.
2. SDK를 쓰기 권한이 있는 폴더에 풋니다. 예: `C:\dev\flutter`.
3. 사용자 PATH에 SDK의 `bin`을 추가하고 새 터미널을 엽니다.
4. Visual Studio Installer에서 **Desktop development with C++** 워크로드와 기본 구성 요소를 설치합니다.
5. `flutter doctor -v`의 Windows·Visual Studio 항목, `flutter devices`의 windows 장치를 확인합니다.

Flutter SDK는 Dart를 포함합니다. Visual Studio의 C++ 빌드 도구는 VS Code 확장과 별도로 필요합니다. 참고: [Flutter SDK 수동 설치](#), [Windows 개발 환경](#).

필요하면 제공한 보조 스크립트로 C++ 도구 설치를 시작할 수 있습니다. 설치 창에서 요구하는 관리자 승인은 직접 처리합니다.

```
powershell -ExecutionPolicy Bypass -File .\scripts\install_windows_build_tools.ps1
```

이 개발 작업공간은 `.tooling/flutter`와 `.tooling/pub-cache`를 사용하며 시스템 PATH를 바꾸지 않았습니다. 스크립트 없이 로컬 SDK를 직접 쓰려면 다음 명령을 사용할 수 있습니다.

```
$env:PUB_CACHE = Join-Path (Get-Location) '.\tooling\pub-cache'
.\tooling\flutter\bin\flutter.bat pub get
powershell -ExecutionPolicy Bypass -File .\scripts\prepare_windows_plugins.ps1
.\tooling\flutter\bin\flutter.bat pub get
.\tooling\flutter\bin\flutter.bat analyze
.\tooling\flutter\bin\flutter.bat test --reporter expanded
```

`.tooling/` 전체는 SDK·캐시·설치 도구이므로 제출에서 제외합니다.

4. 빌드·실행·검사 명령

압축을 푼 뒤 `pubspec.yaml`이 있는 프로젝트 루트에서 실행합니다. 다음은 압축 해제 위치가 `C:\projects\today_todo`인 경우의 예시입니다. 자신의 경로로 바꿉니다.

```
Set-Location -LiteralPath 'C:\projects\today_todo'
```

일반 Flutter 설치 환경에서는 아래 명령을 사용합니다.

```
flutter pub get
powershell -ExecutionPolicy Bypass -File .\scripts\prepare_windows_plugins.ps1
flutter pub get
dart format lib test test_driver scripts
flutter analyze
flutter test --reporter expanded
flutter run -d windows --no-pub
```

Inspector는 debug 실행에 연결합니다. Performance·Timeline·Memory 분석에는 profile 모드를 사용합니다.

```
flutter run -d windows --profile --no-pub
flutter build windows --release --no-pub
```

Windows 플러그인 준비 스크립트는 설치된 패키지 폴더를 junction으로 연결합니다. Developer Mode나 시스템 보안 설정을 바꾸지 않고 관리자 권한도 요구하지 않습니다. 연결은 생성 폴더 `windows/flutter/ephemeral/` 안에 두며 의존성 소스를 복사하지 않습니다. 최초 pub get이 symlink 지원 안내로 끝나더라도 패키지 목록이 생성됐다면 준비 스크립트 후 pub get을 다시 실행합니다. `run_app.ps1`은 이 조건에서 한 번 자동 재시도합니다. 다른 의존성 오류는 먼저 해결합니다. 성공 후 run/build에 `--no-pub`를 사용합니다.

일반 release 빌드 출력은 `build/windows/x64/runner/Release/`이고 제공하는 최종 실행용 복사본은 `output/windows/Release/`입니다. 해당 폴더의 `today_todo.exe`를 실행합니다. 개발 실행에서는 hot reload가 가능하지만, 실제 재시작 보관 검증은 **프로세스를 완전히 종료한 후 새로 실행하여** 확인합니다.

Android는 선택적으로 지원합니다. Android Studio·SDK·에뮬레이터 또는 실제 장치를 준비한 뒤 확인합니다. Windows가 이번 제출의 네이티브 검증 대상입니다.

```
flutter doctor -v
flutter create --platforms=android --project-name today_todo .
flutter pub get
flutter devices
flutter run -d <android-device-id>
flutter run -d <android-device-id> --profile
flutter build apk --release
```

`<android-device-id>`는 실제 장치 ID로 바꿉니다. 플랫폼 생성 명령은 제출에서 제외한 Gradle wrapper 등 생성 파일을 다시 준비하기 위한 명령입니다.

5. 파일 구조와 상태 흐름

경로	역할
lib/main.dart	ProviderScope, MaterialApp, Material 3 테마와 시작 화면
lib/models/todo.dart	제목·카테고리·장소·예상시간·날짜·점수 검증
lib/models/day_plan.dart	날짜별 기준 개수·시간·평점 목표 스냅샷
lib/models/day_reflection.dart	잘한 점·아쉬운 점·개선할 점과 빈 회고 구분
lib/models/place_stats.dart	카테고리·장소별 표본·완료 평균·관찰 성공률
lib/models/planner_state.dart	불변 항목·날짜 계획·설정·고유 ID, 통계·JSON
lib/models/planner_stats.dart	선택 날짜와 최근 기록일의 통계 자료
lib/models/calendar.dart	로컬 날짜 정규화·yyyy-MM-dd 키 검증
lib/models/model_json.dart	저장 JSON 자료형 검사
lib/repositories/todo_repository.dart	로컬 SharedPreferences 저장과 테스트용 메모리 저장
lib/providers/todo_provider.dart	AsyncNotifier, 저장·변경 작업의 순서 관리
lib/screens/todo_screen.dart	입력·선택 날짜·목록·대화상자, 비동기 화면 상태
lib/widgets/todo_tile.dart	장소·분·완료 여부·점수·수정·삭제를 보여주는 항목
lib/widgets/planner_calendar.dart	월 이동·날짜 선택·날짜별 활동 점수
lib/widgets/planner_summary.dart	품질 평균·하루 점수·기준 달성률·기록 평균
lib/widgets/planner_dialogs.dart	평점·항목 수정·목표·점수 설명·ReflectionDialog
lib/widgets/suggestion_text_field.dart	RawAutocomplete로 선택 카테고리의 장소 후보 입력
lib/widgets/place_stats_dialog.dart	장소별 관찰 통계와 카테고리 필터
lib/widgets/planner_style.dart	공통 색상·패널 스타일
test/	계산·상태·저장·화면 동작 테스트
test_driver/	별도 캡처·검증 진입점
scripts/	실행·Windows 플러그인 연결·촬영·PDF·제출 압축 도구
docs/	제품 정책·점검표·스크린샷·실제 검증 로그
windows/	네이티브 runner와 CMake 설정

모델과 로컬 저장

Todo는 다음 자료를 저장합니다. 점수는 정수이며 null은 미완료, 0은 완료한 0점입니다.

```
int id;
String title;
String category;
String location;
int estimatedMinutes;
DateTime date;
int? score;
```

PlannerState는 List<Todo>, Map<String, DayPlan>, Map<String, List<String>> placeCatalog, Map<String, DayReflection> reflections, 하루 목표 개수·시간과 nextId를 보관합니다. 컬렉션은 읽기 전용이며 변경할 때 새 목록·새 상태를 만듭니다. 제목이 같아도 ID로 삭제·수정합니다. nextId도 저장해 재시작 후 ID 중복을 방지합니다.

저장 형식은 version 2 JSON이며 SharedPreferencesAsync의 today_todo_planner_v2 키를 사용합니다. 첫 실행은 빈 상태로 시작합니다. 항목·카테고리·장소 카탈로그·회고·날짜별 기준·목표·설정을 함께 저장합니다. 이전 version 2 자료에 새 필드가 없으면 category는 기타, 회고는 빈 값으로 읽고 카탈로그는 기존 항목의 장소로 보완하여 기록을 유지합니다. 잘못된 JSON·버전·ID·날짜는 읽기 오류로 처리하고 사용자 기록을 자동 삭제하지 않습니다. 기기나 사용자 프로필 변경, 앱 데이터 삭제까지 복원하는 클라우드 백업 기능은 없습니다.

Riverpod 보너스 구현

앱의 ProviderScope가 상태 범위를 제공합니다. AsyncNotifierProvider<TodoNotifier, PlannerState>가 저장소를 읽고, 화면은 ref.watch(todoProvider)로 loading-data-error 상태를 구독합니다. 버튼 동작은 ref.read(todoProvider.notifier)로 변경 명령을 호출합니다.

```
Future<bool> addTodo(String title, {
  String location = '미지정',
  String category = '기타',
  int estimatedMinutes = 30,
  DateTime? date,
});
Future<bool> editTodo(int id, {
  required String title,
  required String location,
  required int estimatedMinutes,
  required DateTime date,
  String? category,
});
Future<void> deleteTodo(int id);
Future<void> rateTodo(int id, int score);
Future<void> reopenTodo(int id);
Future<void> updateGoals((required int count, required int minutes));
Future<void> saveReflection(DateTime date, {
  required String good,
  required String needsWork,
  required String improve,
});
```

추가·수정·삭제 요청은 순서대로 저장합니다. 저장이 성공한 뒤 AsyncData(next)를 공개하여 화면을 갱신합니다. 저장 실패를 성공으로 표시하지 않으며 후속 요청이 이전 요청을 덮어쓰지 않게 합니다. 빈 제목 추가는 false로 거부합니다. 없는 ID 삭제는 상태를 바꾸지 않습니다.

위젯과 생명주기

TodoScreen은 Riverpod 구독과 입력 컨트롤러·선택 날짜·대화상자·저장 중 표시 같은 화면 상태를 함께 다루기 위해 ConsumerStatefulWidget을 사용합니다. 기록 데이터는 provider에, 일시적인 입력과 표시 상태는 화면에 둡니다. TextEditingController, FocusNode, 스크롤 컨트롤러는 소유한 화면에서 dispose합니다.

화면의 전체 스크롤은 CustomScrollView와 SliverToBoxAdapter로 구성하고, 폭에 따라 Row 또는 Column으로 목록·달력·요약을 배치합니다. TodoTile은 Card, Checkbox, Wrap으로 항목 정보를 표시합니다. PlannerCalendar는 월 이동과 날짜 선택을 직접 구현했고, PlannerSummary는 하루 점수·목표·품질 평균·개수·시간 달성률·최근 14기록일을 표시합니다. 공통 패널은 PlannerPanel입니다.

평점·수정·목표·공식 설명·회고·장소별 기록은 대화상자로 구현합니다. 카테고리는 TextField와 공부·운동·기타 chips로 입력합니다. 장소는 RawAutocomplete 기반 SuggestionTextField에서 선택 카테고리의 기존 후보를 선택하거나 새 값을 입력합니다. 화면은 AsyncValue.when으로 loading·error·data를 나누고 변경 요청을 await합니다. 실패하면 입력·기록·대화상자를 유지하며 Snackbar로 알려 줍니다. 전체 테스트 56개 중 화면 테스트 20개가 장소 추천·두 목록 이동·회고 저장/실패·날짜 경계와 이 흐름을 확인합니다.

카테고리·장소 관찰과 회고

카테고리별 placeCatalog는 입력한 장소를 중복 없이 기억합니다. 앞뒤 공백과 연속 공백을 정리하고 빈 카테고리는 기타, 빈 장소는 미지정으로 처리합니다. 미지정은 자동완성 목록에 넣지 않습니다. 할 일을 삭제해도 기존 장소 후보는 남습니다.

장소별 관측 성공률은 **오늘까지의 같은 카테고리·장소 항목 중 8점 이상 완료한 수 / 전체 항목 수**입니다. 미완료는 분모에 포함하고 미래 계획은 제외합니다. 성공 수와 표본 n, 완료한 항목만의 평균점수·완료 예상분을 함께 표시합니다. n이 0이면 기록 없음, 완료가 0개면 평균 없음입니다. 예측 확률이나 가장 좋은 장소를 확정하는 값이 아니며 항목 삭제·편집으로 표본이 바뀔 수 있습니다.

선택 날짜는 미완료인 해야 할 일과 완료한 실제로 한 일로 나눕니다. 완료 취소 시 원래 목록으로 돌아옵니다.

DayReflection은 good·needsWork·improve 세 문자열을 날짜별로 저장합니다. 세 칸을 모두 비우고 저장하면 회고를 제거합니다. 회고만 작성한 날은 달력에 메모 아이콘을 표시하지만 DayPlan을 만들거나 0점 활동 기록·14기록일 평균에 넣지 않습니다.

장소 자동완성·관찰 분모/분자·두 목록 이동·회고 날짜 변경은 전체 자동 테스트에서 확인했습니다. 실제 Windows 종료/재시작에서도 입력한 항목·공부/도서관·평점·세 회고 필드가 복원되는 것을 확인했습니다.

6. 하루 점수와 목표 계산

완료 평점 평균은 완료한 일의 점수 평균입니다. **하루 활동 점수**는 품질과 완료한 양을 함께 반영합니다. 세부 정책과 설계 예시는 docs/product_spec.md에 있습니다.

N = 해당 날짜에 저장된 기준 개수
M = 해당 날짜에 저장된 기준 예상시간(분)
개수 점수 = $\min(10, \text{완료한 일의 점수 합} / N)$
시간 점수 = $\min(10, \text{완료한 일의 (점수} \times \text{예상시간) 합} / M)$
하루 활동 점수 = $(\text{개수 점수} + \text{시간 점수}) / 2$

기본 하루 기준은 3개·90분입니다. 날짜에 첫 계획을 추가할 때 사용자 설정으로 기준을 저장하고, 계획이 늘면 현재 계획량과 비교하여 기준을 올립니다. 삭제·완료 취소·예상시간 감소로 기준을 낮추지 않습니다. 목표 설정 변경은 새로 계획하는 날짜부터 적용합니다.

같은 점수 8점·30분 항목 세 개를 모두 완료하면 평균과 하루 점수는 8점입니다. 두 개만 완료하면 평균은 8점이지만 하루 점수는 약 5.33점입니다. 미완료 항목을 삭제해도 기준 3개·90분이 남아 하루 점수가 올라가지 않습니다. 이는 설계 계산 예시이며 실제 측정 로그와 구분합니다.

낮은 점수의 완료 항목을 삭제하면 평균 수행점수는 높아질 수 있습니다. 활동 점수는 고정되거나 줄어듭니다. 0점 항목 삭제나 각 요소의 상한 적용 상태에서는 점수가 유지될 수 있습니다. 예상시간이 긴 일의 점수는 시간 요소에 더 크게 반영하므로 활동 점수가 단순 품질 평균보다 클 수도 있습니다. 점수·예상시간을 편집하는 경우에는 결과가 달라질 수 있으므로 모든 조작 가능성을 없앴다고 설명하지 않습니다.

다음 새 기록일의 목표는 이전 **최대 14개 기록 날짜의 활동 점수 평균 + 0.5**, 상한 10점입니다. 이전 기록이 없으면 7점입니다. 목표를 처음 저장할 때 그 날짜 자신과 미래 날짜는 제외합니다. 한 번도 계획하지 않은 빈 날은 평균에서 제외하지만, 계획했으나 완료가 없는 날이나 모든 항목을 삭제한 기록일은 0점으로 포함합니다. 기존 날짜의 목표는 그대로 보관합니다. 평균이 10점이면 목표는 최고점 유지입니다.

화면은 소수점 한 자리로 표시하고 계산은 반올림 전 값으로 수행합니다. 예상시간은 사용자가 입력한 계획 시간이며 실제 집중시간 측정값은 아닙니다. 앱 재시작 시 날짜별 기준·점수 자료도 함께 복원합니다.

7. 앱 사용과 기능 스크린샷

1. 달력에서 날짜를 선택합니다. 이전·다음 달 버튼으로 이동하고 오늘 버튼으로 돌아옵니다.
2. 제목·카테고리·장소·예상시간을 입력하여 할 일을 추가합니다. 공부·운동 또는 직접 작성한 카테고리를 쓰고 기존 장소 후보를 선택하거나 새 장소를 입력합니다. 빈 카테고리는 기타, 빈 장소는 미지정, 시간은 1~1,440분입니다.
3. 항목 체크박스를 누르고 0~10점의 수행점수를 선택하여 완료합니다. 0점도 완료입니다.
4. 완료한 항목의 평점 수정으로 점수를 바꾸거나 체크를 해제하여 미완료로 되돌립니다.
5. 수정으로 제목·장소·시간·날짜를 바꾸고, 삭제 버튼으로 해당 ID의 항목을 제거합니다.
6. 하루 점수·완료 평점 평균·완료 개수·시간과 최근 기록일 평균을 비교합니다. 계산 설명 버튼에서 공식과 기준을 확인합니다.
7. 하루 목표 설정을 바꾸면 새로 기록을 만드는 날짜에 적용됩니다.
8. 해야 할 일에는 미완료, 실제로 한 일에는 완료 항목을 표시합니다. 점수 0도 완료 목록에 들어갑니다.
9. 장소별 기록을 열어 카테고리별 관찰 성공률과 표본·완료 평균·완료 예상분을 확인합니다.
10. 날짜별 회고에서 잘한 점·미흡했던 점·다음에 개선할 점을 직접 쓰고 저장합니다. 다른 날짜와 재시작 후에도 같은 내용을 조회합니다.
11. 종료 후 다시 실행하고 같은 날짜를 선택하면 저장된 목록·카테고리별 장소·회고를 확인할 수 있습니다.

최종 Windows debug 네이티브 앱에서 기능 14장을 촬영했습니다. 추가·삭제 대상 행과 장소·회고·두 목록까지 보이며 14장 원본 모두 시각 검토를 통과했습니다. 대상 잘림이나 이전 프레임 잔상이 없고 추가 재촬영은 필요하지 않았습니다. 촬영 완료는 2026-10-07 14:02:45 KST입니다.

파일	화면·증빙	실제 상태
docs/screenshots/01_list.png	제목·장소·분·완료 여부가 있는 목록	최종 촬영·원본 검토 완료
docs/screenshots/02_add_input.png	제목·장소·시간을 입력한 추가 폼	최종 촬영·원본 검토 완료
docs/screenshots/03_add_result.png	새 항목이 목록에 반영된 결과	최종 촬영·원본 검토 완료
docs/screenshots/04_delete_before.png	삭제할 항목과 삭제 전 개수	최종 촬영·원본 검토 완료

파일	화면·증빙	실제 상태
docs/screenshots/05_delete_after.png	해당 항목만 사라진 결과	최종 촬영·원본 검토 완료
docs/screenshots/10_rating_dialog.png	0~10 수행점수 입력 대화상자	최종 촬영·원본 검토 완료
docs/screenshots/11_daily_score.png	평균·하루 점수·개수·시간 기준	최종 촬영·원본 검토 완료
docs/screenshots/12_calendar_history.png	과거 날짜 선택과 기록·점수	최종 촬영·원본 검토 완료
docs/screenshots/13_edit_dialog.png	장소·시간·날짜 수정	최종 촬영·원본 검토 완료
docs/screenshots/14_goal_settings.png	개수·시간 목표 설정과 적용 설명	최종 촬영·원본 검토 완료

기능 증빙 촬영은 동일한 앱에서 목록 → 입력 → 추가 후 → 삭제 전 → 삭제 후 순서로 진행합니다. 이후 평점 입력·하루 점수·달력·수정·설정을 찍습니다. Windows의 Win+Shift+S로 앱 창을 캡처하고 위 경로에 원본으로 저장합니다. 추가·삭제 전후에는 같은 항목을 비교할 수 있도록 제목과 개수를 기록합니다.

촬영용 과거 날짜는 메모리 저장소의 샘플 자료입니다. 실제 로컬 보관 증명은 별도 검증 키에서 **실제 입력 → 앱 종료 → 재실행 → 같은 자료 확인**으로 진행합니다. 최종 검증 로그에는 사용한 키와 동작 범위, 복원한 항목·점수·날짜·기준을 기록합니다.

최종 실제 기능 관찰: 오늘 목록 3개에서 DevTools 화면 캡처·컴퓨터실·45분을 추가하여 4개가 된 것을 확인했습니다. ID 11의 Riverpod 구조 정리를 삭제한 뒤 3개가 남았고 기준은 4개·135분으로 유지됐습니다. 이후 Flutter 강의 복습 30분·8점과 README 작성 20분·9점을 완료하여 품질 평균 8.5점, 하루 활동 점수 3.7점을 확인했습니다. 계산은 $(17/4 + (8 \times 30 + 9 \times 20)/135)/2 = \text{약 } 3.6806$ 이며 화면은 한 자리로 반올림합니다. 과거 날짜 선택과 수정·목표 대화상자도 실제 네이티브 화면으로 촬영했습니다.

완료 2개/기준 4개, 완료 예상시간 50분/기준 135분도 확인했습니다. **해야 할 일은 1개, 실제로 한 일은 2개**로 나뉩니다. 공부·도서관의 관측 성공률은 67%(8점 이상 완료 2개/전체 3개), 완료 평균 7.7점·완료 예상시간 90분입니다. 세 회고 필드와 카테고리별 장소 후보도 실제 화면에 나타납니다.

근거는 docs/evidence/feature_capture_v2.json의 status completed, 화면 점수·표본 읽기와 PNG 14장입니다. 촬영은 2026-10-07 14:02:09~14:02:45 KST에 수행했고 14장 원본의 시각 검토를 통과했습니다. 실제 저장 복원은 별도 검증 키로 확인했으며 9절에서 설명합니다.

최종 추가 증빙 네 장도 촬영·원본 검토를 완료했습니다.

파일	확인할 내용	상태
docs/screenshots/15_place_catalog.png	공부·운동·사용자 카테고리의 장소 자동완성	최종 촬영·원본 검토 완료
docs/screenshots/16_place_stats.png	카테고리·장소별 성공 수/표본·완료 평균·완료 예상분	최종 촬영·원본 검토 완료
docs/screenshots/17_day_reflection.png	잘한 점·미흡했던 점·개선할 점 입력·저장	최종 촬영·원본 검토 완료

파일	확인할 내용	상태
docs/screenshots/18_separate_lists.png	해야 할 일과 실제로 한 일의 분리	최종 촬영·원본 검토 완료

촬영용 과거 3일의 공부·운동, 도서관·집·스터디카페·그린헬스장·레드헬스장 기록은 주입한 샘플입니다. 이 샘플 화면과 아래 실제 로컬 저장 검증을 구분합니다.

8. DevTools 촬영 방법과 보너스

Inspector: debug 앱

1. 4절의 의존성·플러그인 준비 후 `flutter run -d windows --no-pub` 또는 기본 실행 스크립트로 debug 앱을 실행합니다.
 2. 터미널에 표시한 DevTools 링크를 브라우저로 엽니다. 연결이 필요하면 해당 실행의 VM Service 주소를 사용합니다.
 3. Inspector에서 앱 위젯을 선택합니다. 목록 항목·제목·달력·요약 패널의 부모·속성·크기를 확인합니다.
 4. 선택한 위젯과 속성이 함께 보이도록 `06_devtools_inspector.jpg` 또는 동일 stem의 png로 저장합니다.
- VM Service 주소는 실행마다 달라집니다. 주소에 포함된 토큰을 공유용 문서나 이미지에 노출하지 않습니다.

Timeline·Performance: profile 앱

1. debug를 종료하고 `flutter run -d windows --profile --no-pub`로 실행합니다.
2. 이 실행의 DevTools에 연결하여 Performance에서 기록을 시작합니다.
3. 항목 추가·수정·평점·삭제와 달력·스크롤을 실제 수행하고 기록을 멈춥니다.
4. 프레임을 선택하여 프레임 차트와 UI-raster 작업을 관찰합니다. `09_devtools_performance.jpg`로 저장합니다.
5. Timeline Events 또는 이벤트 상세 영역에서 앱 조작 구간의 이벤트를 선택하여 `07_devtools_timeline.jpg`로 저장합니다.

DevTools 2.60.0에서는 Timeline 이벤트를 Performance 화면 안에서 볼 수 있습니다. 사용한 메뉴명과 선택한 이벤트를 기록합니다. 앱의 Timeline 표시는 저장 성공 뒤 상태 공개 구간이며 디스크 저장 전체 시간으로 해석하지 않습니다. UI 검색에서 커스텀 이벤트를 찾지 못하면 보이는 프레임 이벤트로 관찰하고 검색 결과도 그대로 적습니다.

선택 프레임 시간과 한 프레임의 jank 안내를 전체 평균 성능이나 모든 동작의 성능으로 일반화하지 않습니다. 캡처용 드라이버·동시 빌드 등 부하가 있으면 관찰 환경에 적습니다.

Memory: profile 앱

1. 동일 profile 앱의 Memory 화면에서 그래프를 관찰합니다.
2. 목록·완료·달력 동작 전후로 힙과 클래스 인스턴스를 비교합니다.
3. 필요하면 수동 GC와 Refresh 후 Todo·TodoTile 등 인스턴스 수를 확인합니다.
4. 그래프와 표가 보이도록 `08_devtools_memory.jpg`로 저장합니다.

관찰한 시간·동작·클래스 수를 기록합니다. 짧은 관찰만으로 메모리 누수가 없다고 단정하지 않습니다. 이미지에서 잘린 숫자를 화면에 보였다고 쓰지 않으며, 접근성 정보나 원시 VM 자료로 따로 확인했다면 보조 관찰로 구분합니다.

화면·파일	v2 실제 관찰 상태
Inspector · <code>06_devtools_inspector.jpg</code>	최종 Windows debug 연결, ProviderScope → TodayTodoApp·가로1165.3/세로722.7 확인
Timeline · <code>07_devtools_timeline.jpg</code>	todo.delete 검색1/10, Dart 이벤트25μs, 14:09:30 KST

화면·파일	v2 실제 관찰 상태
Memory · 08_devtools_memory.jpg	GC-Refresh 후 Todo10/TodoTile10·힙16.7MB, 14:11:29 KST
Performance · 09_devtools_performance.jpg	Frame942·UI0.7ms-raster8.0ms/phase8.1ms, 14:10:54 KST

Inspector 원본에서는 ProviderScope 아래 TodayTodoApp을 선택한 트리와 width 1165.3, height 722.7을 확인했습니다. 캡처 파일 기록 시각은 2026-10-07 14:03:34 KST입니다.

최종 제출 문서에는 **기능 14장과 DevTools 4장, 총 18장**을 포함합니다. 모든 원본의 시각 검토에서 값·대상 행·잘림을 확인했고 재촬영이 필요한 이미지는 없었습니다.

실제 profile 관찰과 해석

Windows native profile 앱에 DevTools 2.60.0을 연결해 20개 추가·10개 삭제를 완료하고 10개가 남은 것을 UI에서 확인했습니다. 화면/스크롤 요청 115회, 시나리오 24,971ms를 기록했습니다. 드라이버 검증의 warm-up frame이 동작 후 프레임 을 제어하므로 이 세션을 일반 앱의 평균 FPS 측정으로 해석하지 않습니다.

Timeline은 공식 Performance/Timeline Events 화면에서 todo.delete를 검색하여 1/10을 선택했습니다. Dart 범주의 선택 span은 **25μs**입니다. 앱은 저장 성공 후 상태를 공개하는 동기 구간에 Timeline 표식을 남기므로 이 값은 삭제·디스크 저장·화면 렌더 전체 시간이 아닙니다.

원시 Ring 기록 32,398개 중 todo.add는 시작/끝 레코드 12개(6개 span), todo.delete는 20개(10개 span)가 남았습니다. 시작과 끝을 각각 세며 오래된 기록은 덮어쓰일 수 있습니다. 따라서 남은 추가 span 6개를 전체 추가 횟수로 읽지 않습니다. 추가 20회·삭제 10회는 별도 workload UI 검증으로 확인했습니다.

Performance는 **Frame 942**의 tooltip에서 UI 0.7ms-raster 8.0ms, 상세 phase에서 raster 8.1ms-Paint 0.1ms를 확인했습니다. 반올림과 표시 구간에 따라 tooltip과 phase 값이 다릅니다. 차트의 144Hz 기준에서 Raster Jank Detected가 표시됐습니다. 평균 106FPS도 화면에 표시되지만 드라이버 입력·warm-up frame의 영향을 받은 이 세션 값입니다. 이 한 프레임이나 표시 평균을 일반 사용 성능으로 일반화하지 않습니다.

Memory에서는 수동 GC와 Refresh 후 **Todo 10개·TodoTile 10개**, Dart heap 16.7MB(원시값 17,476,352B)를 확인했습니다. GC 이전 보조 관찰은 Todo20·TodoTile49·30.3MB였습니다. 이는 특정 시점의 allocation profile 비교이며 메모리 누수가 없다는 증명은 아닙니다. 클래스 수와 현재 목록 10개가 일치한 사실을 기록했습니다.

시각·모드·선택 이벤트·프레임·해석은 docs/evidence/devtools_observations.json에 있습니다. 실제 동작은 profile_workload.json, 원시 이벤트 표본은 timeline_check.json에서 확인합니다.

9. 확장 버전의 실제 검증 결과

아래는 카테고리·장소 관찰·두 목록·회고까지 포함한 최종 소스 검사와 네이티브 검증 상태입니다.

항목	v2 상태·근거
의존성 설치	성공, docs/evidence/v2_pub_get.txt
정적 분석	원본22.8초·깨끗한 복사본16.5초·최종 보조 도구18.1초, 모두 No issues found!
자동 테스트	전체 56개 통과, 26.0초, v2_flutter_test.txt
Windows debug	최종 빌드21.4초·기능14장·점수·두 목록·장소·회고 확인, 원본 검토 통과
Windows profile	최종 빌드94.3초·실제 연결·20추가/10삭제/10잔존·115회 화면/스크롤 요청 확인

항목	v2 상태-근거
일반 release 빌드-실행	빌드111.5초·exit0, 배포 exe 실행-응답True 확인
로컬 저장 재시작	create/read 모두 passed, persistence_create.json·persistence_read.json
기능-확장 이미지	최종 14장 완료-전체 원본 검토 통과, feature_capture_v2.json
DevTools 이미지	최종4장 완료-전체 원본 검토 통과, devtools_observations.json
깨끗한 소스 복사본	pub get-analyze-56개 test-release 성공, v2_clean_build.json
최종 PDF-ZIP	PDF31페이지-캡처18장 시각 검토, ZIP 필수 파일-CRC-해시-의존성 제외 검사 통과

전체 **56개(모델-상태-저장 36개 + widget 20개)** 테스트가 통과했습니다. 날짜-불변 상태-0점 완료-활동량-삭제 후 기준 유지-14기록일-목표 스냅샷·JSON 왕복-순차 저장-실패 복구와 카테고리-카탈로그-관찰 표본-두 목록-회고-입력-좁은 화면을 확인합니다. 저장소 테스트는 메모리 저장소의 JSON과 오류 동작 검사이며 실제 Windows 디스크 재시작 검증과 구분합니다.

실제 Windows 쓰기 검증에서는 재시작 저장 검증-공부-도서관-45분-9점을 입력하고 세 회고 필드를 저장했습니다. 2026-10-07 13:39:04 KST에 쓰기를 확인하고 프로세스를 종료한 뒤, 같은 별도 검증 키로 새 프로세스를 실행했습니다. **13:56:16 KST에 복원 검증도 통과했습니다.** 다시 읽은 화면은 1개-완료 평균 9.0-하루 점수 3.8-공부/도서관/45분/9점-관측률 100%(1/1)이었으며 세 회고 필드도 같은 내용을 확인했습니다. 근거는 persistence_create.json과 persistence_read.json의 status passed, 읽기 검증 exit 0입니다.

검증 진입점은 별도의 저장 키에서 일반 SharedPreferencesRepository를 사용합니다. 드라이버 화면 검사를 위한 warm-up frame 보조를 포함하며 실제 로컬 저장 자료를 새 프로세스에서 읽었습니다. 장소 관측 100%는 표본 1개의 결과이고 다른 장소나 앞으로의 수행을 예측하지 않습니다.

기록 위치는 docs/evidence/입니다. v2 자동 검사는 v2_pub_get.txt, v2_flutter_analyze.txt, v2_flutter_test.txt, 기능 촬영은 feature_capture_v2.json에 있습니다. 최종 요약은 validation_v2.json, 쓰기-복원은 persistence_*.json, DevTools는 devtools_observations.json에 기록합니다. 제출 복사본 로그도 최종 결과로 갱신합니다. 기존 v1 로그의 값을 v2 성공 증거로 사용하지 않습니다.

깨끗한 소스 복사본 .tooling/v2-native-build-check/run-20261007-133951에서 최종 앱 코드가 현재 소스와 일치함을 확인했습니다. 의존성 설치 성공, 분석 문제 없음 16.5초, 전체 테스트 56개 통과(리포터 28초), release 빌드 111.5초를 확인했습니다. 명령 시작부터 끝까지의 기록 시간은 각각 분석 24.399초·테스트 49.430초·빌드 116.337초로, 도구가 표시한 작업 시간과 구분합니다. 근거는 v2_clean_analyze.txt, v2_clean_test.txt, v2_clean_release.txt, v2_clean_build.json입니다.

생성한 Release 폴더의 exe·DLL·data 등 11개 파일을 output/windows/Release/로 복사하고 원본과 SHA256이 모두 같은지 확인했습니다. 확인 시각은 2026-10-07 13:46:11 KST입니다. 이 폴더는 실행용이며 코드 제출 ZIP에는 넣지 않습니다. ZIP의 파일 구성·CRC-작업공간/복사본/압축 payload 일치와 검증한 소스 71개 해시를 확인했습니다. 근거는 package_check_v2.json입니다.

최종 배포 exe를 2026-10-07 14:12:24 KST에 시작했고 14:12:40 KST에 Today Todo - Daily Planner 창과 응답 상태 true를 확인했습니다. 실행 위치는 output/windows/Release/today_todo.exe, 근거는 release_launch_v2.json입니다. 기능 조작은 debug/profile의 실제 UI와 테스트로, release는 빌드-배포-시작으로 검증했습니다.

최초 실행 보조 검증은 새 소스 폴더에서 Windows PowerShell 5.1로 수행했습니다. 첫 pub get의 symlink 오류 후 manifest를 확인하여 junction을 만들고 한 번 재시도해 12.175초·exit 0으로 의존성 준비를 완료했습니다. PowerShell 오류 처리 설정도 복원했습니다. 이 검사는 실행 스크립트의 의존성 준비 구간이며, 빌드-앱 시작은 별도 검증입니다. 근거는 v2_first_run_check.json·v2_first_run_check.txt, 완료 시각은 2026-10-07 13:55:47 KST입니다.

제출 증빙은 v2 자동 검사-현재 기능 캡처·profile/Memory/Timeline-실제 저장 복원-최종 패키지 기록만 선별합니다. scripts/clean_submission_paths.py는 검사 수치와 결과를 유지하며 프로젝트 절대경로를 상대경로로 바꾸어 문서와 로그

를 다른 컴퓨터에서도 읽기 쉽게 합니다.

10. 코드 제출 포함·제외

포함하는 자료는 다음과 같습니다.

- lib/, 작성한 test/와 test_driver/.
- pubspec.yaml, pubspec.lock, analysis_options.yaml, .metadata, .gitignore.
- 필요한 windows/와 Android 플랫폼 소스·빌드 설정.
- 자체 scripts/, 앱 실행.cmd, README와 제품 정책·점검표.
- 실제 docs/screenshots/, 검증 근거, 최종 Readme.pdf.

제외하는 자료는 .tooling/, build/, .dart_tool/, SDK, 패키지 캐시, 다운로드한 의존성 소스, .git/, IDE 개인 설정, 임시 생성물입니다. 플랫폼의 ephemeral-Gradle 캐시·개인 경로 local.properties·다운로드된 wrapper 실행 파일도 제외합니다. 빌드에 필요한 자체 runner·CMake 설정까지 삭제하지 않습니다. Android 생성 파일 재준비는 4절 명령을 사용합니다.

pubspec.yaml과 pubspec.lock은 의존성 선언·버전 고정 자료입니다. 실제 패키지 소스는 받는 사람이 flutter pub get으로 설치합니다. Release 폴더는 이 컴퓨터에서 바로 실행하기 위한 결과물이며 과제의 자체 코드 ZIP에는 넣지 않습니다.

11. PDF 재생성과 제출 ZIP

스크린샷을 교체하고 실제 관찰을 README에 기록한 뒤 PDF를 다시 만듭니다. 다음 스크립트는 PATH의 Python으로 실행합니다. PATH에 없으면 실제 Python 실행 파일을 -PythonPath 옵션으로 지정할 수 있습니다.

```
powershell -ExecutionPolicy Bypass -File .\scripts\update_readme.ps1
```

일반 Python 환경에서는 필요한 라이브러리를 설치하고 직접 실행할 수 있습니다.

```
python -m pip install reportlab Pillow
python scripts/build_readme_pdf.py
```

출력은 output/pdf/Readme.pdf입니다. Windows의 맑은 고딕 글꼴을 사용하고 기능·DevTools 원본 캡처를 부록에 넣습니다. 가로 원본은 가로 A4, 세로 원본은 세로 A4에 원본 비율을 유지해 배치합니다. 같은 파일 stem의 png와 jpg를 지원합니다.

생성한 PDF는 본문 13페이지와 캡처 부록 18페이지, 총 31페이지입니다. 전 페이지 렌더링에서 한글·명령·표·코드·footer·캡처 가독성과 여백 검사를 통과했습니다. 파일을 수정하면 PDF를 재생성하고 변경 페이지를 다시 검토한 뒤 제출 폴더와 ZIP을 만듭니다.

```
powershell -ExecutionPolicy Bypass -File .\scripts\package_submission.ps1
```

submission/today_todo_생성시각/에 자체 코드와 Readme.pdf가 모이고 같은 이름의 ZIP이 생성됩니다. 제출 ZIP의 필수 구성·최신18장·PDF·CRC·작업공간·복사본/압축 파일 일치와 SDK·캐시·빌드 결과 제외를 확인했습니다. 깨끗한 소스의 분석·56개 테스트·Windows release 빌드와 검증 소스 해시도 일치합니다. 수정 후 압축하면 같은 검사를 반복합니다. 제출 ZIP·PDF를 직접 확인하고 마감 전에 과제 사이트에 업로드합니다.

실제 실행 캡처 | 리스트

Windows 네이티브 앱에서 할 일 3개의 장소, 예상 시간, 미완료 상태를 표시한 화면.

19 20 21 22 23 24 25

26 27 28 29 30 31

날짜 아래 숫자는 하루 점수예요. 오늘

선택한 날의 하루 점수

0.0 / 10

목표 7.5점

평점과 해낸 양을 함께 반영해요.

완료 평점 평균 아직 없음

완료 개수 0 / 3개

0% 달성

완료 예상시간 0 / 90분

0% 달성

📌 점수는 어떻게 계산하나요?

최근 14기록일의 평소 페이스

선택 날짜까지의 최근 4기록일

하루 점수 평균 5.3점

평소 완료 개수 2.3개

평소 하루 예상시간 60분

날짜별 전체 목록 3개

해야 할 일 3개

☐ Flutter 강의 복습

공부 📍 도서관 ⌚ 30분 ○ 미완료

✎ 수정

☐ Riverpod 구조 정리

공부 📍 스테디룸 ⌚ 40분 ○ 미완료

✎ 수정

☐ README 작성

공부 📍 집 ⌚ 20분 ○ 미완료

✎ 수정

실제로 한 일 0개

완료한 일이 아직 없어요. 체크하고 평점을 남겨 보세요.

실제 실행 캡처 | 추가 전 입력

새 제목 "DevTools 화면 캡처", 장소 "컴퓨터실", 예상 시간 45분을 입력한 화면.

<2026년 10월>

월

화

수

목

금

토

일

1

2

3

4

5

6

7

8

9

10

11

12

13

14

15

16

17

18

19

20

21

22

23

24

25

26

27

28

29

30

31

날짜 아래 숫자는 하루 점수예요.

오늘

선택한 날의 하루 점수

0.0 / 10

목표 7.5점

평점과 해낸 양을 함께 반영해요.

완료 평점 평균 아직 없음

완료 개수0 / 3개

0% 달성

완료 예상시간n / 90분

2026년 10월 7일 (수)

계획 3개 · 90분

하루 피드백 쓰기

장소별 기록

새 할 일

DevTools 화면 캡처

+ 추가

카테고리

공부

장소

컴퓨터실

예상 시간

45

분

공부

운동

기타

날짜별 전체 목록

3개

해야 할 일

3개

Flutter 강의 복습

공부

도서관

30분

미완료

수정

Riverpod 구조 정리

공부

스터디룸

40분

미완료

수정

README 작성

실제 실행 캡처 | 추가 결과

DevTools 화면 캡처 항목이 추가되어 전체 목록이 4개가 된 화면.

19 20 21 22 23 24 25

26 27 28 29 30 31

날짜 아래 숫자는 하루 점수예요. 오늘

선택한 날의 하루 점수

0.0 / 10

목표 7.5점

평점과 해낸 양을 함께 반영해요.

완료 평점 평균 아직 없음

완료 개수 0 / 4개

0% 달성

완료 예상시간 0 / 135분

0% 달성

📌 점수는 어떻게 계산하나요?

최근 14기록일의 평소 페이스

선택 날짜까지의 최근 4기록일

하루 점수 평균 5.3점

평소 완료 개수 2.3개

평소 하루 예상시간 68분

날짜별 전체 목록 4개

해야 할 일 4개

☐ Flutter 강의 복습

공부 📍 도서관 ⌚ 30분 ○ 미완료

✎ 수정

☐ Riverpod 구조 정리

공부 📍 스튜디오 ⌚ 40분 ○ 미완료

✎ 수정

☐ README 작성

공부 📍 집 ⌚ 20분 ○ 미완료

✎ 수정

☐ DevTools 화면 캡처

공부 📍 컴퓨터실 ⌚ 45분 ○ 미완료

✎ 수정

시작하기 4개

실제 실행 캡처 | 삭제 전

삭제 대상 "Riverpod 구조 정리"가 포함된 4개 항목.

19202122232425

262728293031

날짜 아래 숫자는 하루 점수예요.오늘

선택한 날의 하루 점수

0.0 / 10

목표 7.5점

평점과 해낸 양을 함께 반영해요.

완료 평점 평균 아직 없음

완료 개수0 / 4개

0% 달성

완료 예상시간0 / 135분

0% 달성

📌 점수는 어떻게 계산하나요?

최근 14기록일의 평소 페이스

선택 날짜까지의 최근 4기록일

하루 점수 평균5.3점

평소 완료 개수2.3개

평소 하루 예상 시간68분

날짜별 전체 목록4개

해야 할 일4개

☐ Flutter 강의 복습

공부📍 도서관 ⌚ 30분 ⚪ 미완료

✎ 수정

☐ Riverpod 구조 정리

공부📍 스테디룸 ⌚ 40분 ⚪ 미완료

✎ 수정

☐ README 작성

공부📍 집 ⌚ 20분 ⚪ 미완료

✎ 수정

☐ DevTools 화면 캡처

공부📍 컴퓨터실 ⌚ 45분 ⚪ 미완료

✎ 수정

시작하기

실제 실행 캡처 | 삭제 결과

"Riverpod 구조 정리"가 삭제되고 다른 3개 항목이 남은 화면. 삭제 후 기준 작업량은 유지된다.

19 20 21 22 23 24 25

26 27 28 29 30 31

날짜 아래 숫자는 하루 점수예요. 오늘

선택한 날의 하루 점수

0.0 / 10

목표 7.5점

평점과 해낸 양을 함께 반영해요.

완료 평점 평균 아직 없음

완료 개수 0 / 4개

0% 달성

완료 예상시간 0 / 135분

0% 달성

📌 점수는 어떻게 계산하나요?

최근 14기록일의 평소 페이스

선택 날짜까지의 최근 4기록일

하루 점수 평균 5.3점

평소 완료 개수 2.3개

평소 하루 예상 시간 60분

날짜별 전체 목록 3개

해야 할 일 3개

☐ Flutter 강의 복습

공부 📍 도서관 ⌚ 30분 ○ 미완료

✎ 수정

☐ README 작성

공부 📍 집 ⌚ 20분 ○ 미완료

✎ 수정

☐ DevTools 화면 캡처

공부 📍 컴퓨터실 ⌚ 45분 ○ 미완료

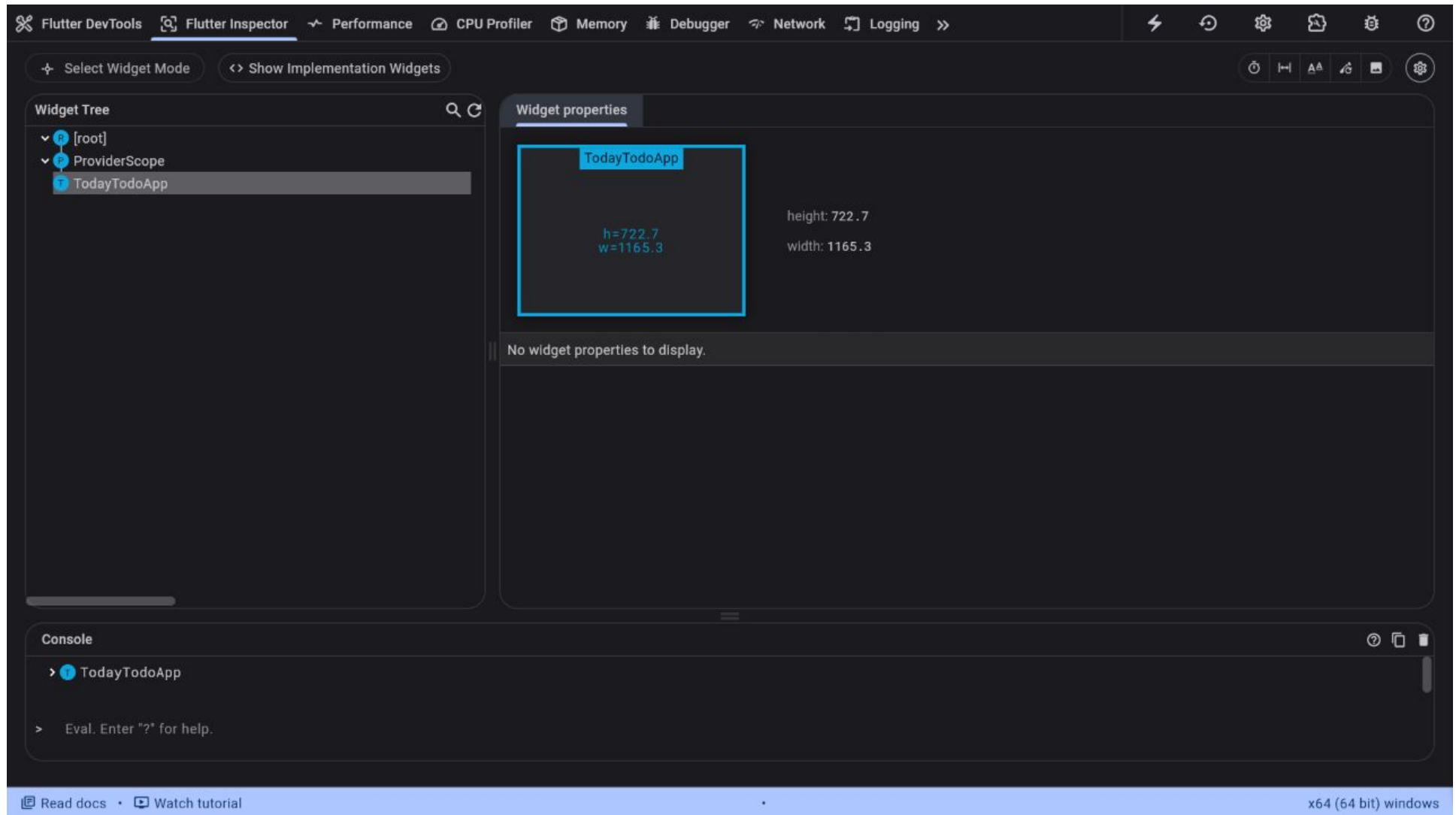
✎ 수정

실제로 한 일 0개

완료한 일이 아직 없어요. 체크하고 평점을 남겨 보세요.

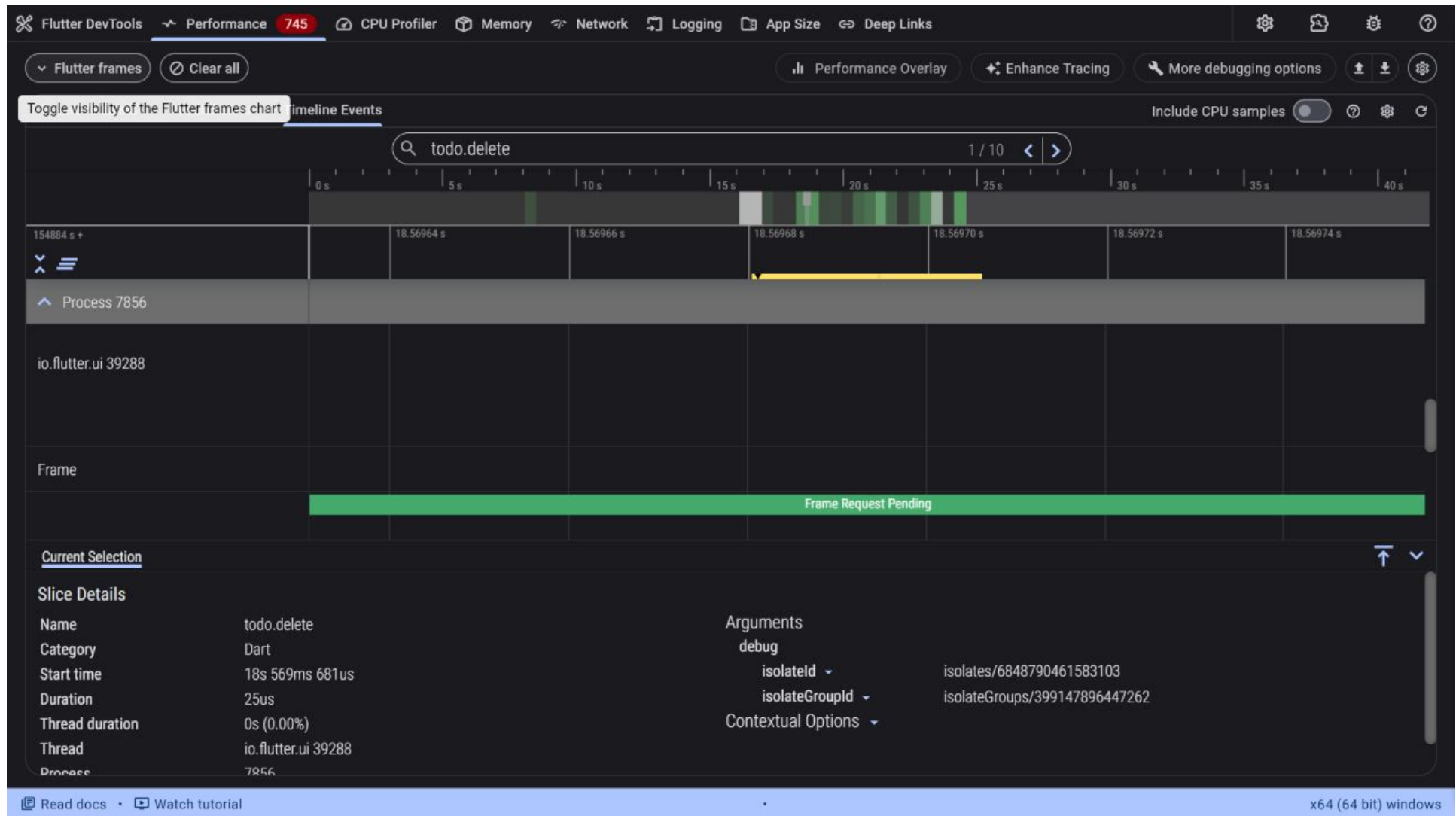
실제 실행 캡처 | DevTools Inspector

debug 네이티브 앱의 위젯 트리와 속성 검사.



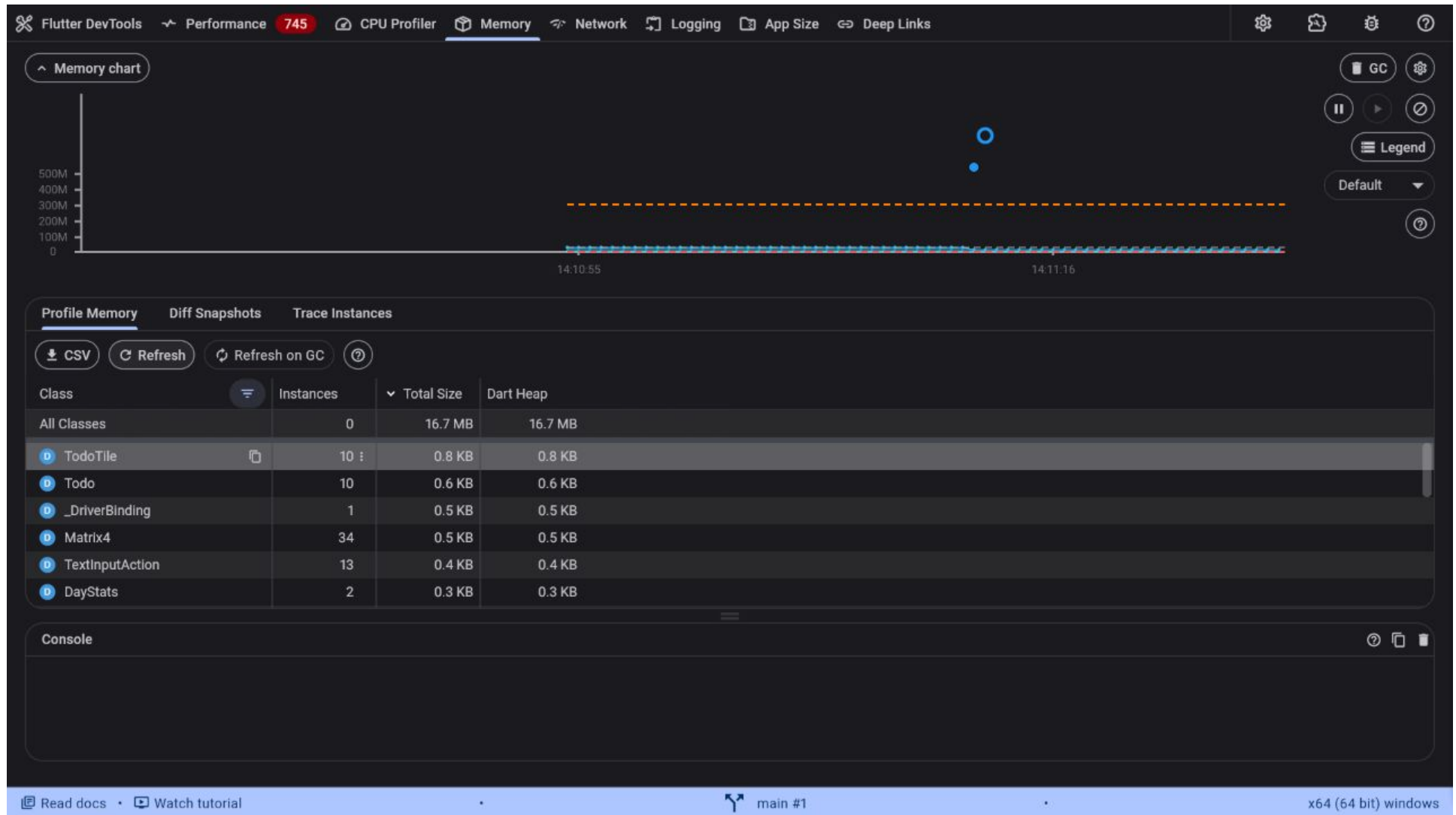
실제 실행 캡처 | DevTools Timeline

Windows profile 앱의 실제 프레임 타임라인과 이벤트 상세 화면.



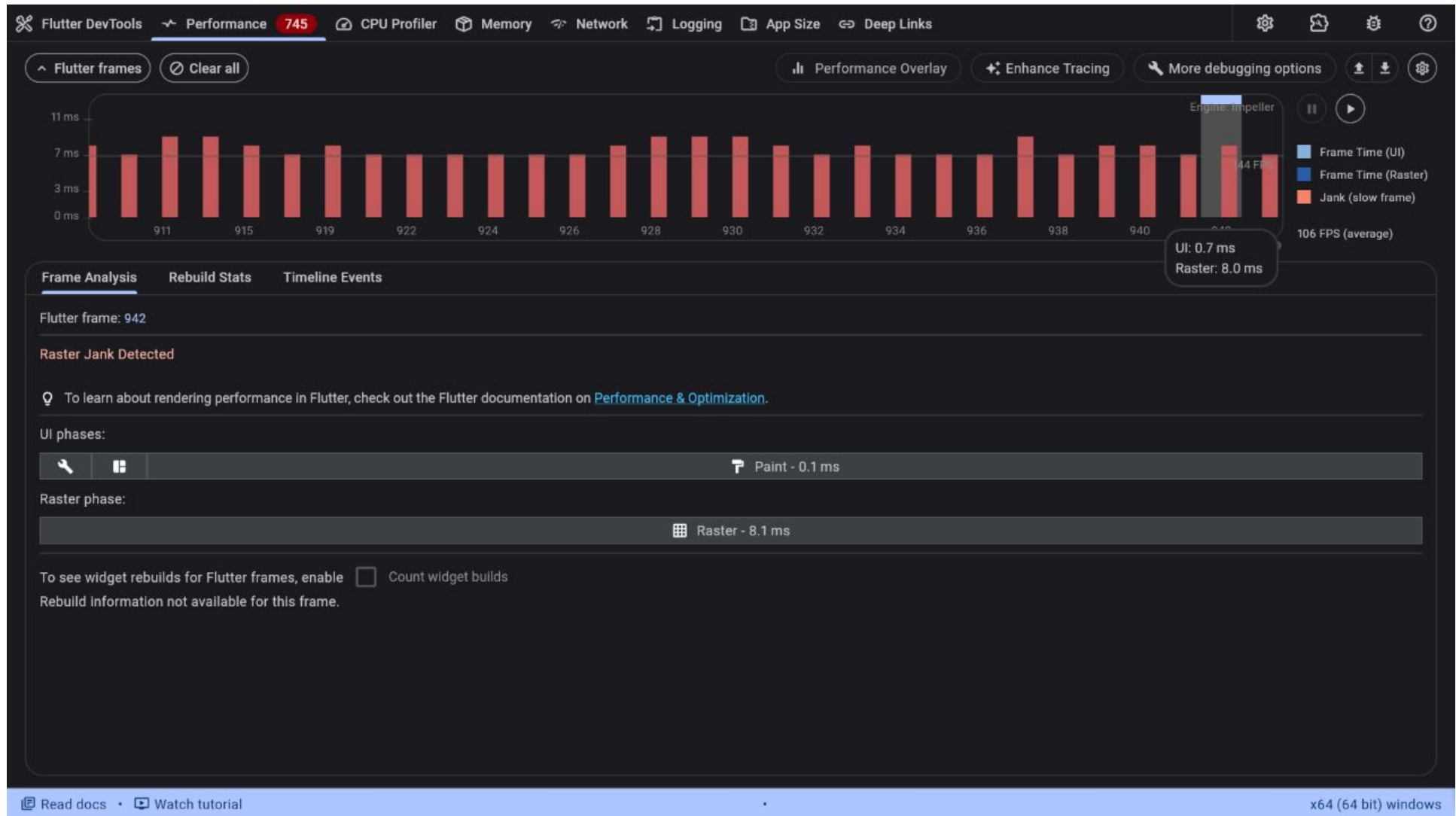
실제 실행 캡처 | DevTools Memory

Windows profile 앱의 실제 메모리 사용량과 객체 할당 관찰 화면.



실제 실행 캡처 | DevTools Performance

Windows profile 앱의 프레임 그래프와 선택 프레임 분석 화면.



실제 실행 캡처 | 완료와 수행 평점

완료 체크 후 0~10점 수행 점수를 입력하는 화면. 0점도 완료 상태로 저장된다.

날짜 아래 숫자는 하루 점수예요.

오늘

선택한 날의 하루 점수

0.0 / 10

목표 7.5점

평점과 해낸 양을 함께 반영해요.

완료 평점 평균 아직 없음

완료 개수 0 / 4개

0% 달성

완료 예상시간 0 / 135분

0% 달성

📌 점수는 어떻게 계산하나요?

최근 14기록일의 평소 페이스

선택 날짜까지의 최근 4기록일

하루 점수 평균 5.3점

평소 완료 개수 2.3개

평소 완료 예상시간 68분

다음 새 기록일 목표 5.8점

☐ Flutter 강의 복습

공부 📍 도서관 ⌚ 30분 🔄 미완료

✎ 수정

☐ README 작성

📍 집 ⌚ 30분 🔄 미완료

☐ ...

0개

완료 평점 남기기

Flutter 강의 복습

이번 할 일을 얼마나 잘 해냈나요?
0점도 완료한 기록으로 남습니다.

완료 평점 8 / 10

취소 평점 저장

0개

완료한 일이 아직 없어요. 체크하고 평점을 남겨 보세요.

실제 실행 캡처 | 하루 점수와 작업량

완료 평점 평균 8.5점과 개수·시간을 반영한 하루 점수 3.7점을 구분해 표시한다.

날짜 아래 숫자는 하루 점수예요.

오늘

선택한 날의 하루 점수

3.7 / 10

목표 7.5점

평점과 해낸 양을 함께 반영해요.

완료 평점 평균 8.5

완료 개수 2 / 4개

50% 달성

완료 예상시간 50 / 135분

37% 달성

📌 점수는 어떻게 계산하나요?

최근 14기록일의 평소 페이스

선택 날짜까지의 최근 4기록일

하루 점수 평균	6.2점
평소 완료 개수	2.8개
평소 완료 예상시간	80분

다음 새 기록일 목표	6.7점
-------------	------

☐ DevTools 화면 캡처

공부 📍 컴퓨터실 ⌚ 45분 ○ 미완료

✎ 수정

실제로 한 일 2개

☑ Flutter 강의 복습

공부 📍 도서관 ⌚ 30분 ★ 완료 · 8/10점

✎ 수정 ☆ 평점 수정

☑ README 작성

공부 📍 집 ⌚ 20분 ★ 완료 · 9/10점

✎ 수정 ☆ 평점 수정

실제 실행 캡처 | 달력 기록

달력에서 지난 날짜를 선택해 완료된 할 일과 날짜별 점수를 조회한다. 촬영용 과거 기록은 예시 자료다.

<2026년 10월>

월

화

수

목

금

토

일

1

2

3

4

5

6

7

8

9

10

11

12

13

14

15

16

17

18

19

20

21

22

23

24

25

26

27

28

29

30

31

날짜 아래 숫자는 하루 점수예요.

오늘

선택한 날의 하루 점수

8.0 / 10

목표 7.0점

평점과 해낸 양을 함께 반영해요.

완료 평점 평균 8.0

완료 개수 3 / 3개

100% 달성

완료 예상시간 90 / 90분

2026년 10월 6일 (화)

계획 3개 · 90분

하루 피드백 쓰기

장소별 기록

새 할 일

+ 추가

카테고리

장소

예상 시간

공부

도서관

30

분

공부

운동

기타

날짜별 전체 목록

3개

해야 할 일

0개

해야 할 일이 없어요. 새 계획을 위에 적어 보세요.

실제로 한 일

3개

☒

강의 복습

🗑

공부

📍 도서관

🕒 30분

★ 완료 · 8/10점

✎ 수정

☆ 평점 수정

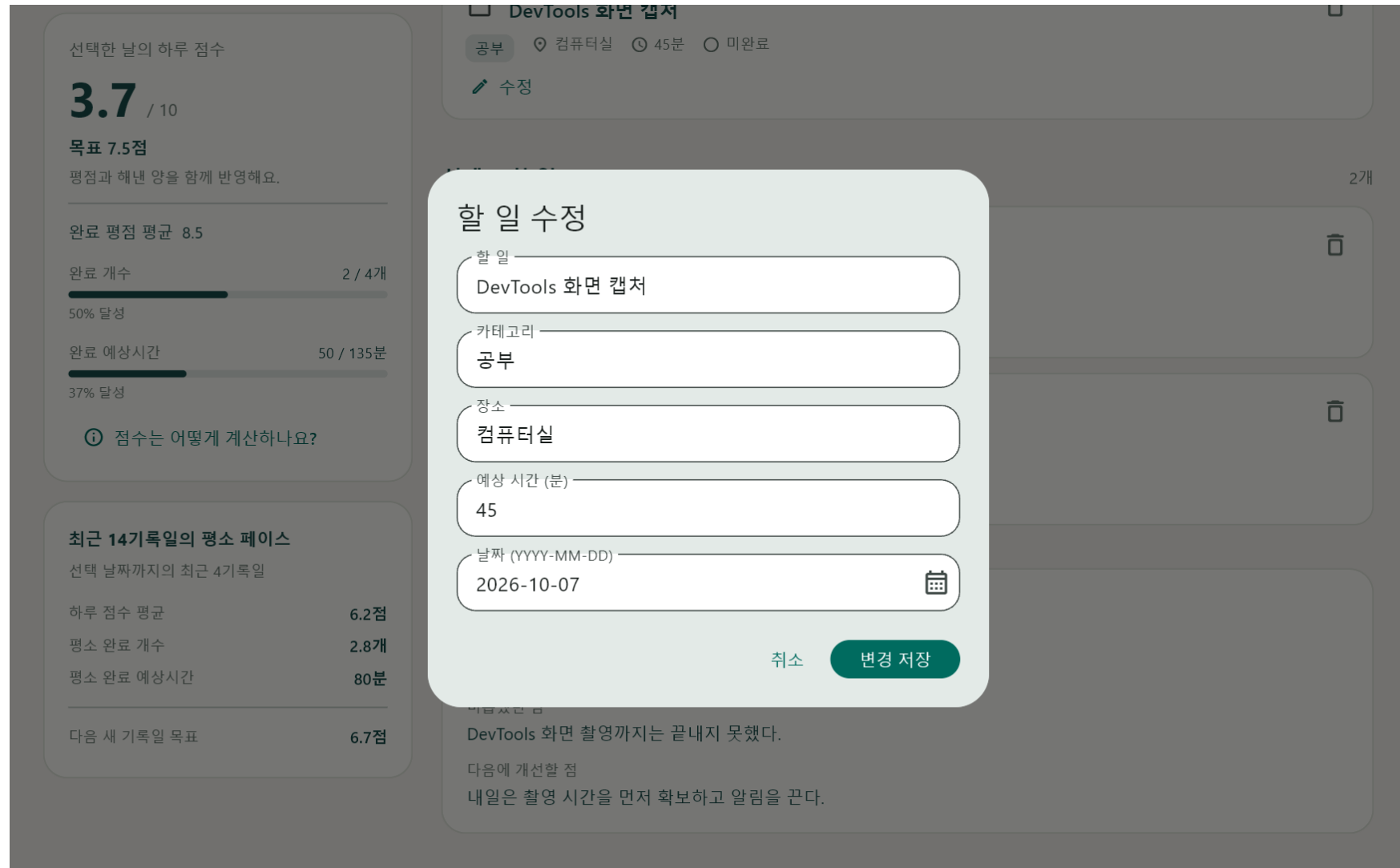
☒

운동

🗑

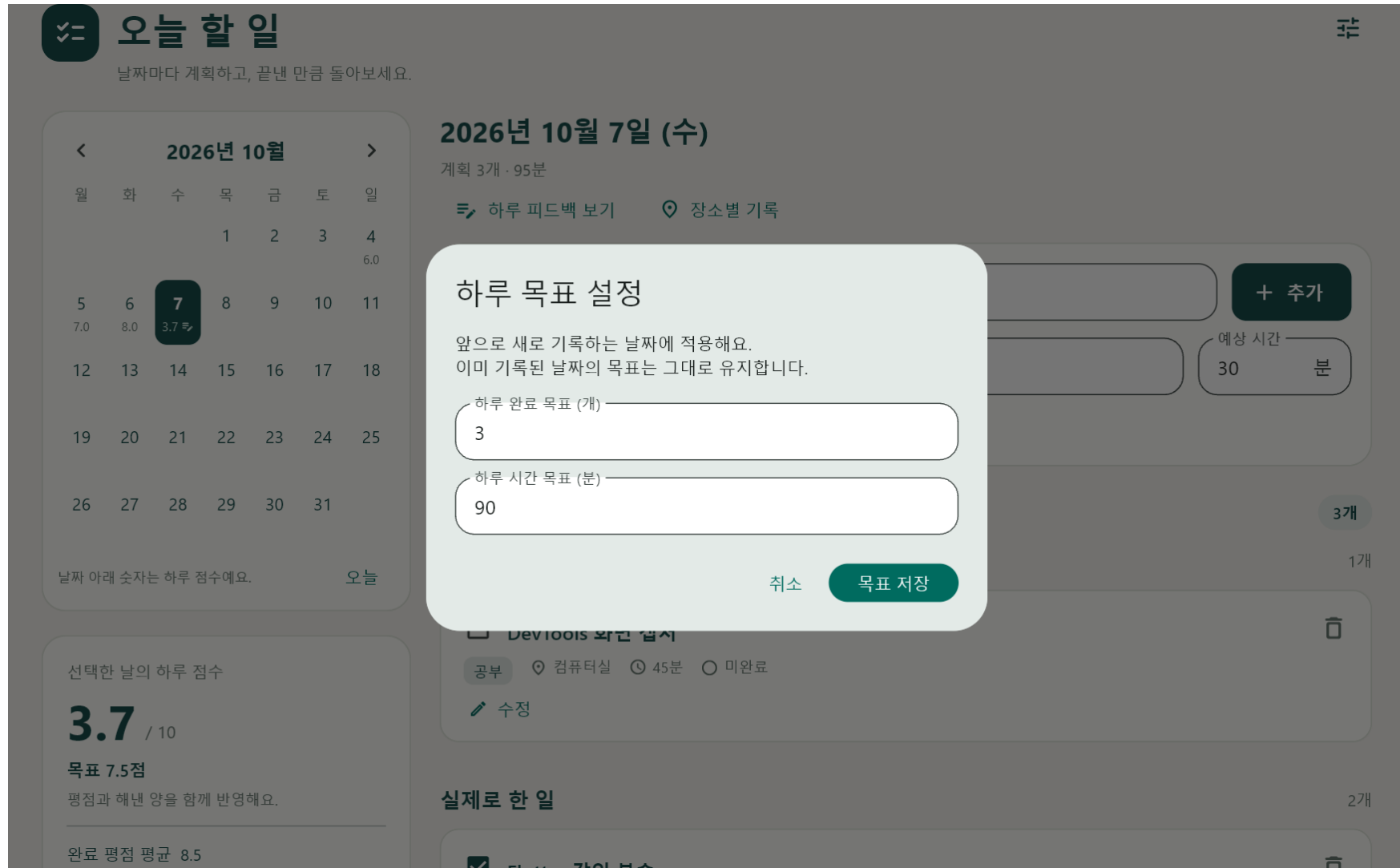
실제 실행 캡처 | 할 일 수정

제목, 장소, 예상 시간, 날짜를 수정하는 화면.



실제 실행 캡처 | 하루 목표 설정

기본 개수·시간 목표를 설정한다. 새 기록일에 적용하고 이미 기록한 날짜의 기준은 유지한다.



실제 실행 캡처 | 입력한 장소 재사용

공부 카테고리에서 사용자가 입력했던 도서관을 자동완성 후보로 표시한다.

12 13 14 15 16 17 18

19 20 21 22 23 24 25

26 27 28 29 30 31

날짜 아래 숫자는 하루 점수예요. 오늘

선택한 날의 하루 점수

3.7 / 10

목표 7.5점

평점과 해낸 양을 함께 반영해요.

완료 평점 평균 8.5

완료 개수 2 / 4개
50% 달성

완료 예상시간 50 / 135분
37% 달성

점수는 어떻게 계산하나요?

최근 14기록일의 평소 페이스

서태 날짜까지의 최근 기록인

카테고리 공부

장소 도서관

예상 시간 30 분

공부 운동 도서관

날짜별 전체 목록 3개

해야 할 일 1개

☐ DevTools 화면 캡처

공부 컴퓨터실 45분 미완료

수정

실제로 한 일 2개

☒ Flutter 강의 복습

공부 도서관 30분 완료 · 8/10점

수정 평점 수정

☒ README 작성

공부 집 20분 완료 · 9/10점

수정 평점 수정

오늘 할 일 · 하루 기록 | Flutter + Riverpod | v2.0

28

실제 실행 캡처 | 카테고리별 장소 기록

장소별 8점 이상 완료 비율, 평균 평점, 표본 수와 완료 예상시간을 표시한다. 미래 계획은 제외한다.

장소별 기록

오늘까지의 기록을 카테고리·장소별로 모았어요.
성공률은 8점 이상 완료한 수 ÷ 전체 할 일 수입니다. 미완료도 포함하고 미래 계획은 제외해요.

적은 표본은 크게 달라질 수 있어요. 이 값은 기록의 관측 비율이며 앞으로의 수행을 예측하지 않습니다.

카테고리
전체 카테고리

공부 · 도서관

관측 성공률 67%

8점 이상 완료 2개 / 전체 3개

완료 평점 평균 7.7

완료 3개 · 완료 예상시간 90분

공부 · 스터디룸

관측 성공률 기록 없음

8점 이상 완료 0개 / 전체 0개

완료 평점 평균 아직 없음

완료 0개 · 완료 예상시간 0분

입력했던 장소는 남아 있지만 오늘까지의 할 일 기록은 없어요.

닫기

실제 실행 캡처 | 직접 쓰는 하루 피드백

잘한 점, 미흡했던 점, 다음에 개선할 점을 직접 작성하고 날짜별로 저장한다.

The screenshot shows a mobile application interface for daily feedback. On the left, there is a calendar grid with the 7th of the month highlighted. Below the calendar, a summary for the selected day shows a score of 3.7 out of 10, with a target of 7.5. Progress bars indicate completion of tasks: 2/4 tasks completed (50%), and 50/135 minutes of estimated time (37%).

The main part of the screen is a form titled '하루 피드백 쓰기' (Write Daily Feedback). It includes fields for '새 할 일' (New Task), '카테고리' (Category), '장소' (Location), and '예상 시간' (Estimated Time). A modal dialog is currently open over the form, titled '하루 피드백' (Daily Feedback). The dialog contains three sections:

- 잘한 점** (What I did well on): 도서관에서 강의 복습을 마쳤고 README도 정리했다. (I finished reviewing the lecture at the library and organized the README.)
- 미흡했던 점** (What I did poorly on): DevTools 화면 촬영까지는 끝내지 못했다. (I couldn't finish taking a screenshot of the DevTools screen.)
- 다음에 개선할 점** (What I will improve next): 내일은 촬영 시간을 먼저 확보하고 알림을 끈다. (Tomorrow, I will secure the recording time first and turn off notifications.)

At the bottom of the modal, there are buttons for '취소' (Cancel) and '피드백 저장' (Save Feedback). The background form also shows a list of tasks, including '공부' (Study) at the library for 30 minutes, with a completion status of 8/10 points.

실제 실행 캡처 | 해야 할 일과 실제로 한 일

미완료와 완료 항목을 별도 목록으로 표시한다. 완료 평점을 저장하면 실제로 한 일 목록으로 이동한다.

날짜 아래 숫자는 하루 점수예요.

오늘

선택한 날의 하루 점수

3.7 / 10

목표 7.5점

평점과 해낸 양을 함께 반영해요.

완료 평점 평균 8.5

완료 개수 2 / 4개

50% 달성

완료 예상시간 50 / 135분

37% 달성

① 점수는 어떻게 계산하나요?

최근 14기록일의 평소 페이스

선택 날짜까지의 최근 4기록일

하루 점수 평균	6.2점
평소 완료 개수	2.8개
평소 완료 예상시간	80분

다음 새 기록일 목표

6.7점

로그인

로그아웃

3기

해야 할 일 1개

☐ DevTools 화면 캡처

공부

📍 컴퓨터실

🕒 45분

○ 미완료

✎ 수정

🗑

실제로 한 일 2개

☒ Flutter 강의 복습

공부

📍 도서관

🕒 30분

★ 완료 · 8/10점

✎ 수정

☆ 평점 수정

🗑

☒ README 작성

공부

📍 집

🕒 20분

★ 완료 · 9/10점

✎ 수정

☆ 평점 수정

🗑